DATA 260 — Homework 8

Kafka request and response, three-agent pipeline, and failure tests

Student: Haozhe Wang

SID: 8561

Entity: Trial

Branch: hw8-work

Short summary

This homework uses Kafka to connect the Trial API and workers. The client sends a request with a correlation ID. The worker uses the same ID in the response. The second part sends one question through Planner, Writer, and Reviewer. The third part tests duplicate delivery, consumer death, poison messages, and message ordering.

Main settings

Item	Value
API port	8461
Kafka prefix	s8561_
Tasks partitions	2
Model	qwen3:8b
Verify seed	268561

Part 1 — Kafka request and response

The client sends CREATE_TRIAL to s8561_trial_requests. The Trial worker reads the message and sends the result to s8561_trial_responses. The client matches the result by correlation_id.

CREATE_TRIAL and GET_TRIAL both worked. The same correlation ID was kept for the full flow:

431e7d65-f255-443c-b5ec-9b3a2bd040ae

The created Trial had a trial_id. GET_TRIAL returned the same Trial. Error responses also keep correlation_id so the client can match errors.

Topics

Use	Topic
Request	s8561_trial_requests
Response	s8561_trial_responses

Part 2 — Three-agent pipeline

The question first enters s8561_inbox. Planner makes a plan and sends it to s8561_tasks. Writer creates a draft and sends it to s8561_drafts. Reviewer checks the draft and sends the final answer to s8561_final.

The full path worked:

s8561_inbox -> s8561_tasks -> s8561_drafts -> s8561_final

The final answer was read from s8561_final by send_question.py. The three processes used qwen3:8b locally.

Agent roles

Agent	Job
Planner	Makes a plan
Writer	Writes a draft
Reviewer	Checks the draft and sends the final answer

Part 3 — Failure tests

The tests show what the application must handle when Kafka delivery is not exactly once.

Test	Observed result
Duplicate before fix	Same correlation ID, but two different trial IDs. same_trial_id was False.
Duplicate after fix	Same correlation ID and same trial ID. The CREATE operation became idempotent.
Consumer death	Writer stopped before commit. After restart, the message was processed again and COMMIT_SUCCES
Poison message	The message was retried 3 times, sent to s8561_dead_letter, and the next good message was committ
Ordering	Without a key, completion order was mixed. With a key, all 20 messages completed from 0 to 19 in one

Ordering result

Without a key: 1, 0, 3, 2, 5, 4, 7, 9, 11, 6, 8, 10, 12, 13, 15, 17, 14, 16, 18, 19.

With a key: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19.

Kafka keeps order inside one partition. A key sends messages for the same entity to the same partition. This improves order, but it can reduce parallel work.

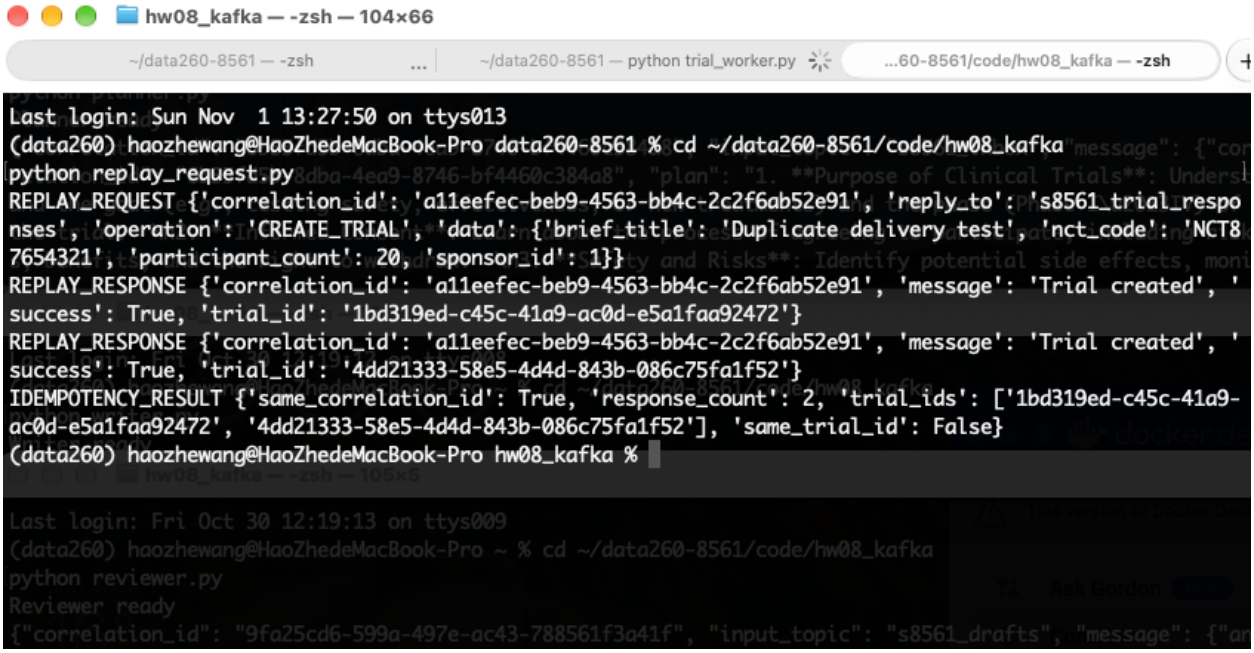
Evidence screenshots

These are the screenshots available in the submitted files. The screenshot checklist records the other expected names.

Part 1 worker ready



Part 3 duplicate before idempotency fix



Part 3 poison message and dead-letter result

